07

백트래킹

하나 고르고, 다음으로 가고, 다시 되돌리기

숫자마다 고를 수 있는 문자가 있습니다

2에서는 a·b·c, 5에서는 j·k·l

입력 "25"는 두 자리입니다.

각 숫자에서 문자 하나씩 고릅니다.

만들 문자열의 길이는 2

전화 키패드 문제의 입력과 답

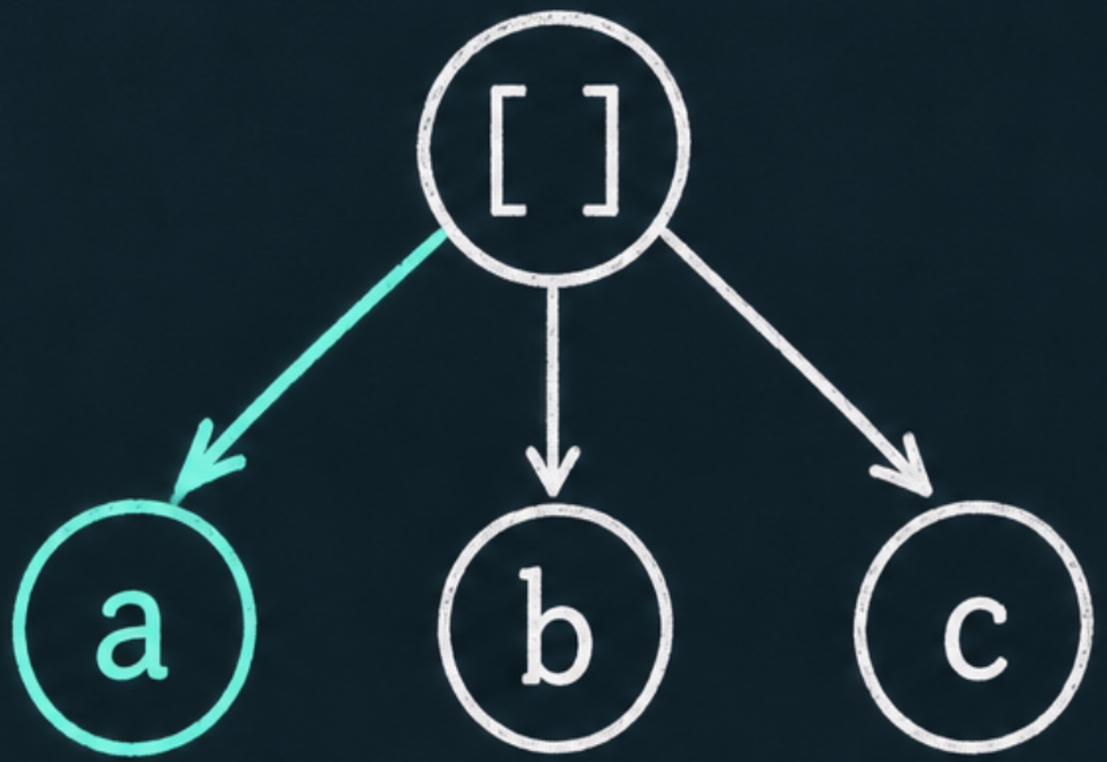
입력: 2~9로 이루어진 문자열, 예: "25"

반환: 만들 수 있는 모든 문자열의 목록

"25" → ["aj", "ak", ..., "cl"]

문자 조합의 개수 9만 반환하는 문제가 아닙니다.

첫 자리에서 갈래가 세 개 생깁니다



아직 아무 문자도 고르지 않았습니다

첫 숫자 2의 후보는 a, b, c입니다.

a를 고른 갈래부터 끝까지 봅니다.

빈 선택 []에서 출발

a를 골랐으면 첫 자리는 그대로 둡니다

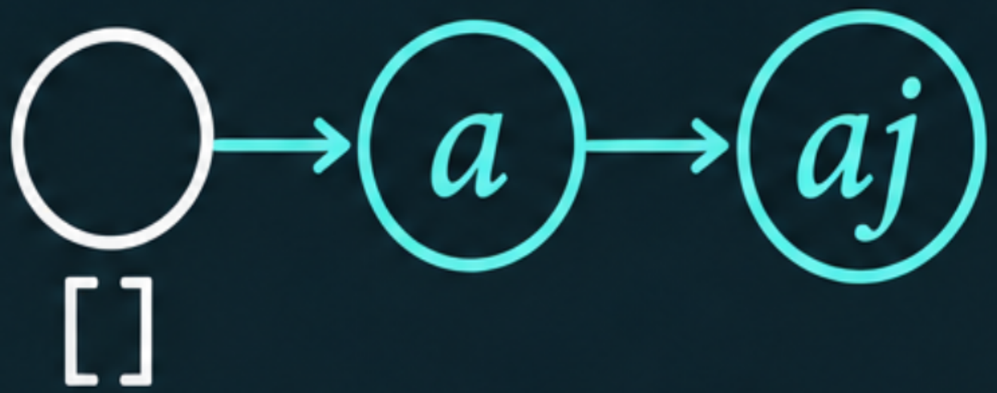
현재 선택은 [a]입니다

첫 자리의 선택이 끝났습니다.

이제 둘째 숫자 5를 봅니다.

다음 후보는 j, k, l

j를 붙이면 aj가 완성됩니다



입력 두 자리를 모두 처리했습니다

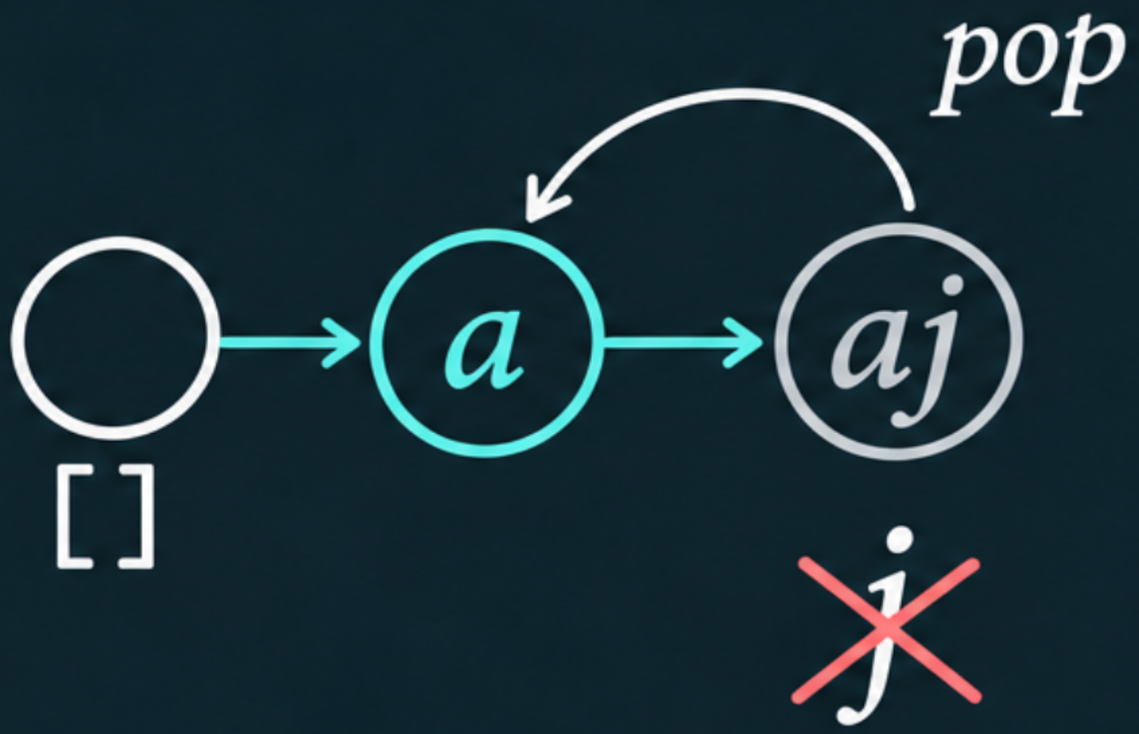
현재 선택은 [a, j]입니다.

완성한 "aj"를 답 목록에 저장합니다.

aj

답 목록에 "aj" 추가

j만 지우고 a로 돌아갑니다



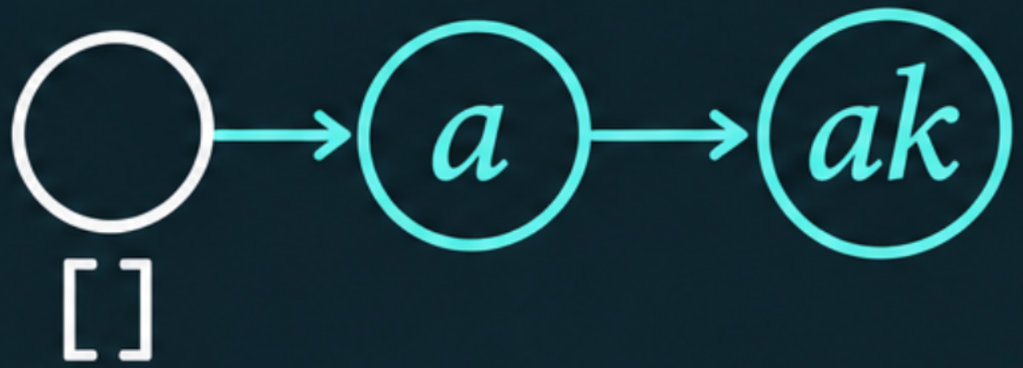
방금 고른 마지막 문자만 취소

저장한 답 "aj"는 남겨둡니다.

현재 선택은 다시 [a]가 됩니다.

이제 k를 고를 수 있습니다.

같은 a 뒤에 이번에는 k를 붙입니다



aj ak

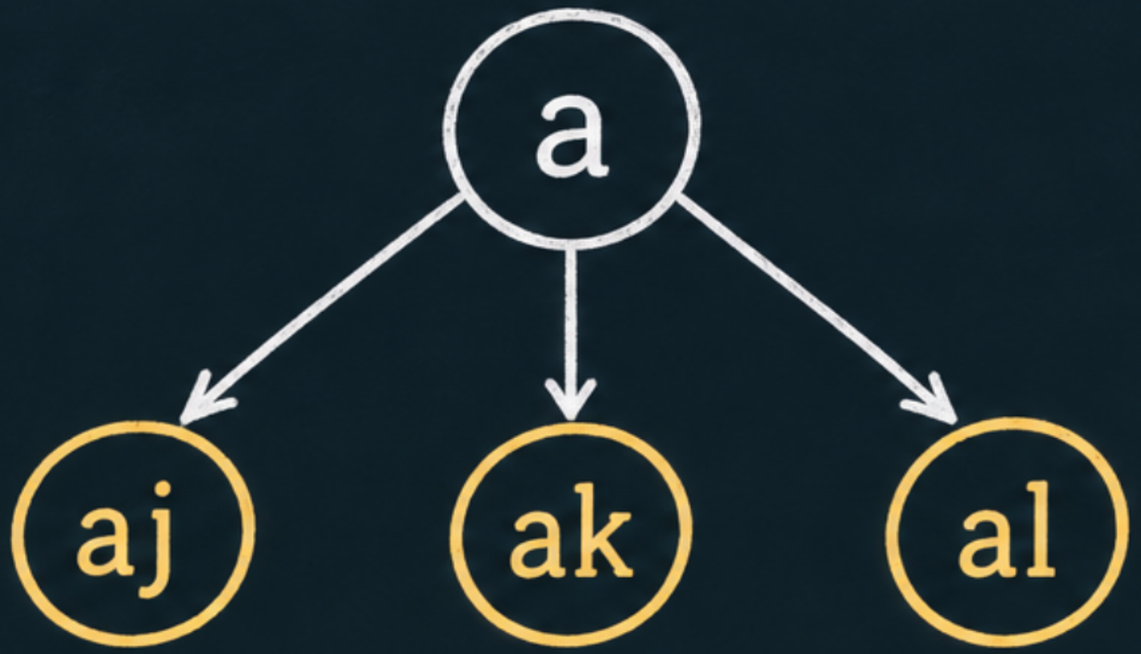
돌아온 자리에서 다음 후보를 선택

현재 선택 [a]에 k를 붙입니다.

"ak"를 답 목록에 추가합니다.

aj, ak, 그리고 al

a 아래의 세 답을 모두 찾았습니다



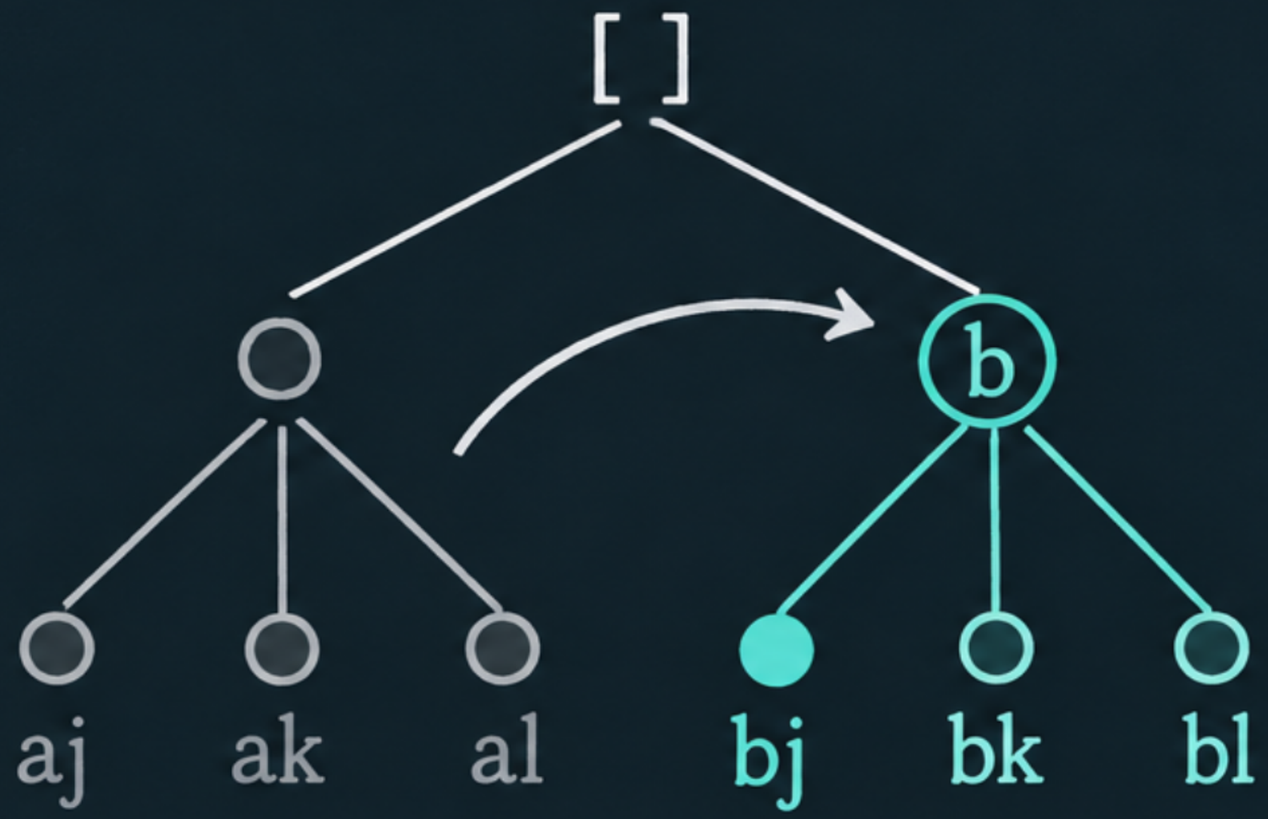
둘째 자리의 후보 j·k·l을 모두 확인

aj, ak, al을 저장했습니다.

이제 첫 자리 a도 취소합니다.

첫 자리의 다음 후보는 b

첫 자리도 돌아와서 바꿉니다



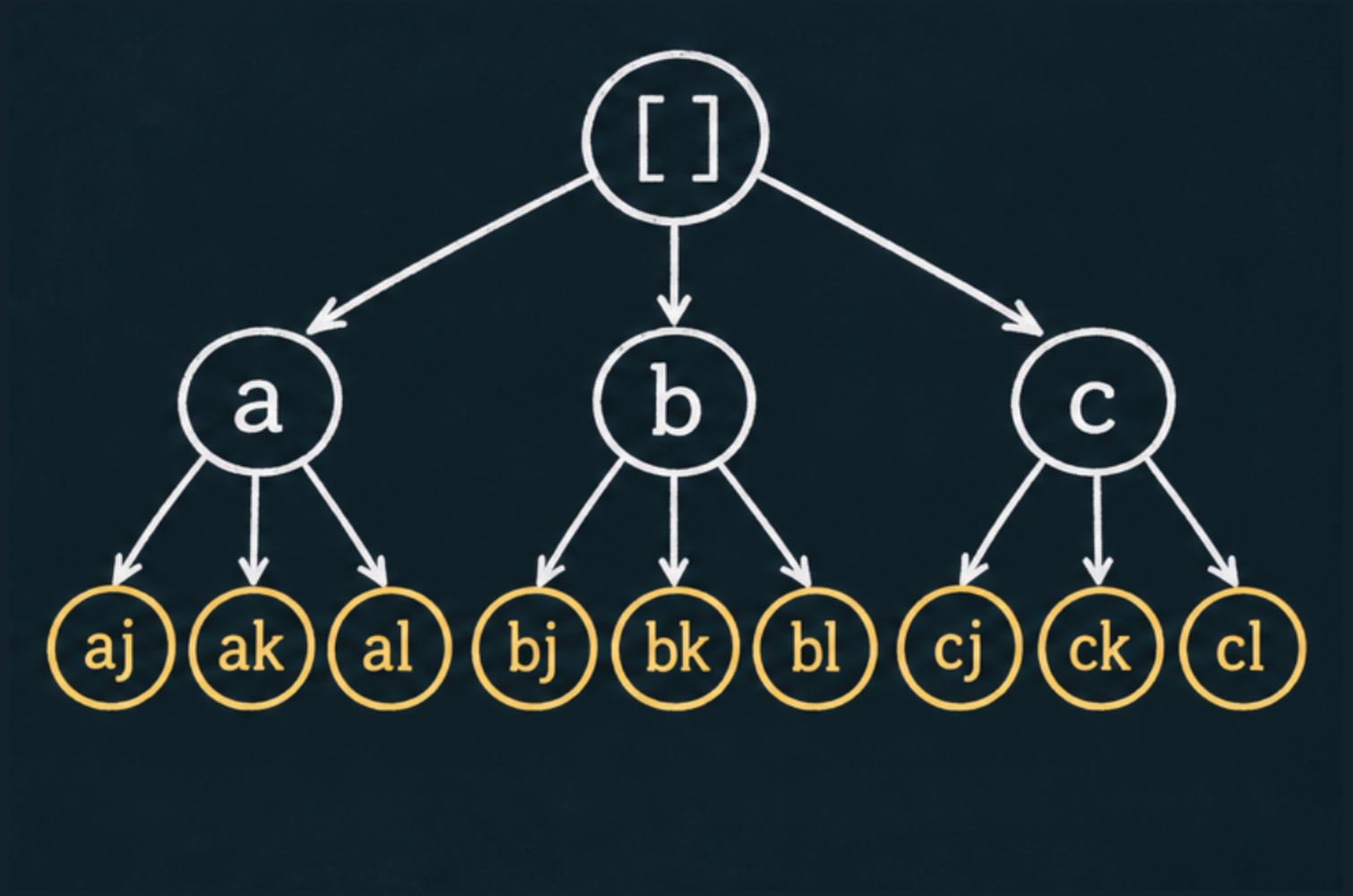
a 갈래를 마치고 b 갈래로 이동

b를 고른 뒤 j부터 다시 시작합니다.

그다음 c 갈래도 같은 방법입니다.

어느 갈래도 빠뜨리지 않기

25에서 만드는 아홉 개의 답



끝에 도착한 갈래 하나가 답 하나

a 아래 3개, b 아래 3개, c 아래 3개

3 × 3 = 9개의 문자열

aj ak al · bj bk bl · cj ck cl

함수에 알려줄 것은 두 가지

i: 다음에 고를 숫자의 위치

path: 지금까지 고른 문자들

i = 1, path = ["a"]

25의 1번 위치는 두 번째 숫자 5입니다.

숫자에 맞는 문자를 읽는 Python 사전

```
keys = {"2": "abc", "5": "jkl", "9": "wxyz"}
keys["2"]    # "abc"
keys["5"]    # "jkl"

for ch in keys["5"]:
    print(ch)  # j, k, l을 하나씩 출력
```

키 "5"로 문자 묶음 "jkl"을 찾습니다.

그림의 움직임을 세 줄로 적으면

```
path.append(ch)
visit(i + 1)
path.pop()
```

문자 붙이기 → 다음 자리 처리 → 붙인 문자 지우기

입력 끝에 도착하면 답을 저장합니다

```
1   if i == len(digits):
2       answers.append("".join(path))
3       return
```

["a", "j"]를 문자열 "aj"로 합쳐 저장

전화 키패드 · Python 전체 코드

```python
def letter_combinations(digits):
    keys = {"2":"abc", "3":"def", "4":"ghi",
            "5":"jkl", "6":"mno", "7":"pqrs",
            "8":"tuv", "9":"wxyz"}
    answers, path = [], []
    def visit(i):
        if i == len(digits):
            answers.append("".join(path))
            return
        for ch in keys[digits[i]]:
            path.append(ch)
            visit(i + 1)
            path.pop()
    if digits:
        visit(0)
    return answers
```

visit(0)에서 출발합니다. path는 모든 호출이 함께 사용합니다.

세 자리도 같은 동작을 한 번 더 합니다

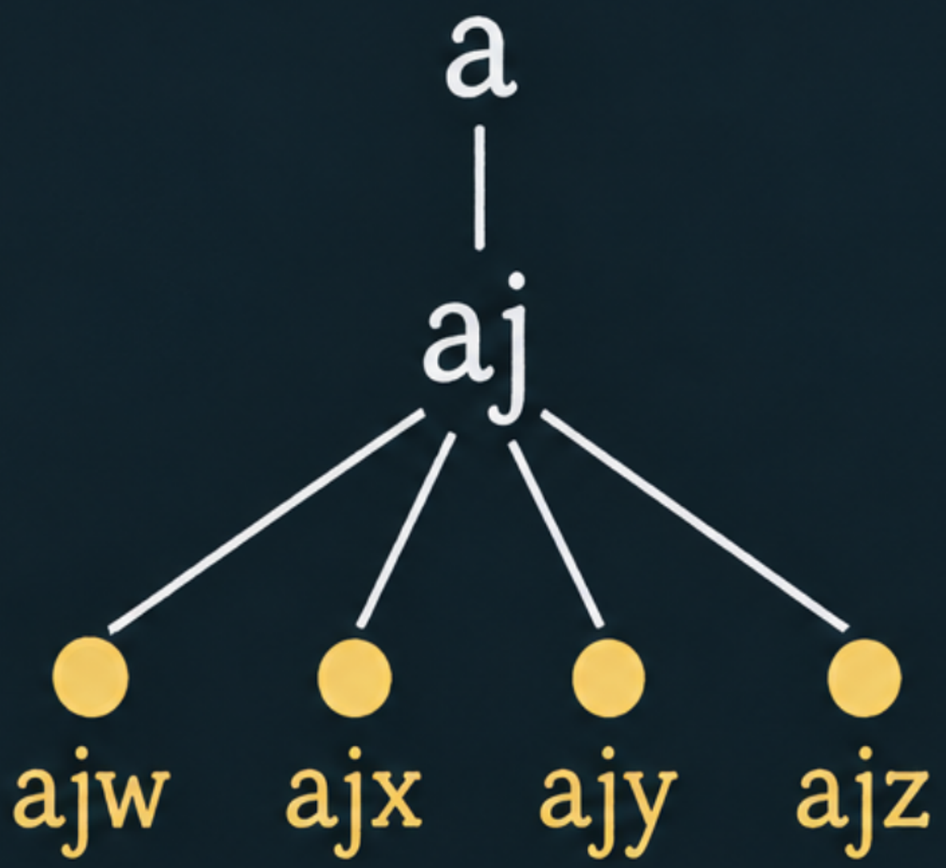
입력 "259"는 세 자리

2: a·b·c / 5: j·k·l

9: w·x·y·z

완성되는 문자열의 길이는 3

aj 뒤에서 마지막 문자만 바꿉니다



ajw를 저장한 뒤 w만 지웁니다.

ajx, ajy, ajz도 같은 방식입니다.

네 후보가 끝나면 j를 지웁니다.

그다음 ak에서 다시 w부터

답이 많으면 만드는 시간도 늘어납니다

25: 3 × 3 = 9개

259: 3 × 3 × 4 = 36개

n자리라면 답은 최대 4^n개

답 저장 시간 $O(n \cdot 4^n)$, 작업 공간 $O(n)$, 답 목록 공간은 별도

6장에서는 답을 저장했고, 여기서는 선택을 바꿉니다

계단: 앞에서 구한 방법 수로 다음 답 계산

전화 문자: 현재 경로를 바꾸며 완성 답 수집

지금 해놓은 선택이 무엇인지 보기

다음 문제에서는 실패가 확실한 갈래를 중간에 멈춥니다.

7-1 · 서로 다른 카드로 두 자리를 채우면

카드 1·2·3에서 두 장을 놓습니다.

한 카드는 두 번 쓸 수 없습니다.

놓는 순서가 다르면 다른 배치입니다.

반환할 값은 배치의 개수

순서를 바꾸면 다른 배치가 됩니다

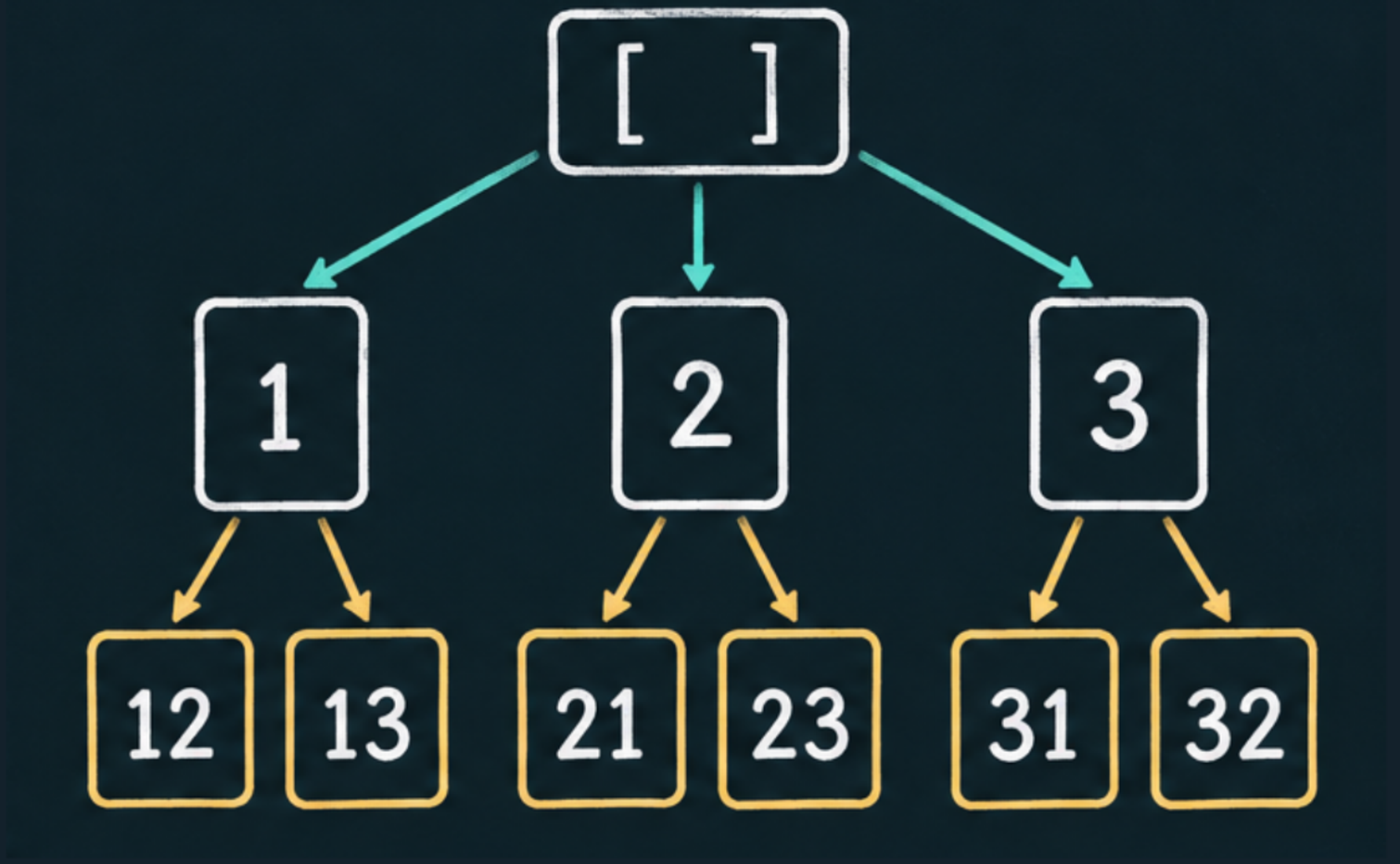
[1, 2]와 [2, 1]은 둘 다 셉니다.

[1, 1]은 같은 카드를 두 번 써서 불가

카드를 되돌려 놓고 다음 배치를 만듭니다.

다른 배치에서 다시 쓰는 것은 가능

끝까지 놓은 갈래가 여섯 개입니다



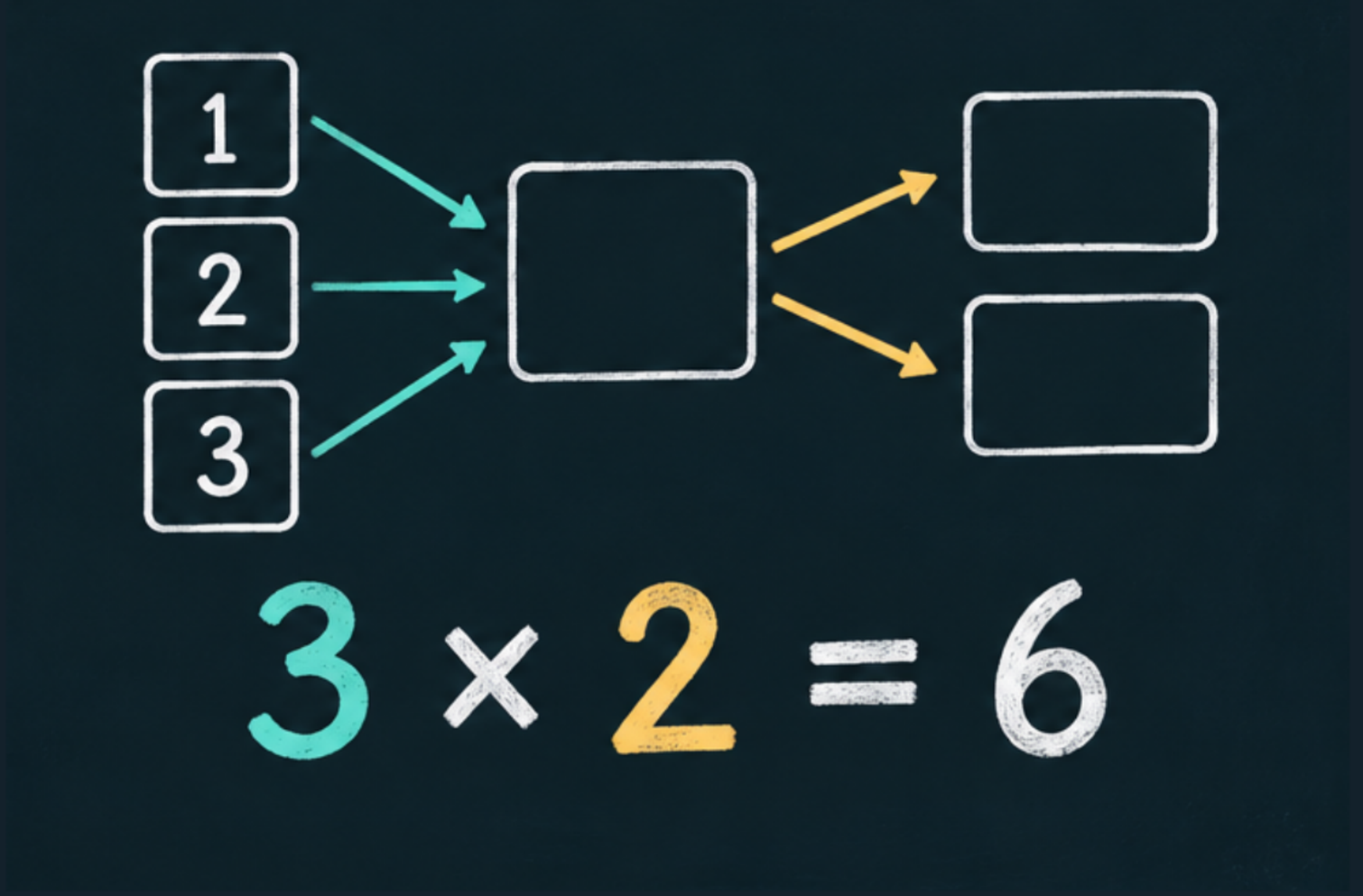
첫 카드마다 남은 두 장 중 선택

1 뒤에는 2 또는 3

2 뒤에는 1 또는 3, 3 뒤에는 1 또는 2

12, 13, 21, 23, 31, 32

개수만 필요하므로 선택 수를 곱합니다



첫 자리 3가지, 다음 자리 2가지

각 첫 선택에서 남은 카드 수가 같습니다.

3 × 2 = 6

n부터 r번, 하나씩 줄여 곱하기

7-1 · Python

```python
def permutation_count(n, r):
    answer = 1
    for i in range(r):
        answer *= n-i
    return answer
```

answer를 1로 시작하고, 각 자리의 선택 수를 곱합니다.

7-1 · C

```
1   int permutation_count(int n, int r) {
2       int answer=1;
3       for (int i=0; i<r; ++i) answer*=n-i;
4       return answer;
5   }
```

int는 정수입니다. i++은 다음 자리로 한 번 이동합니다.

7-2 · 합이 3이 되는 선택을 찾습니다

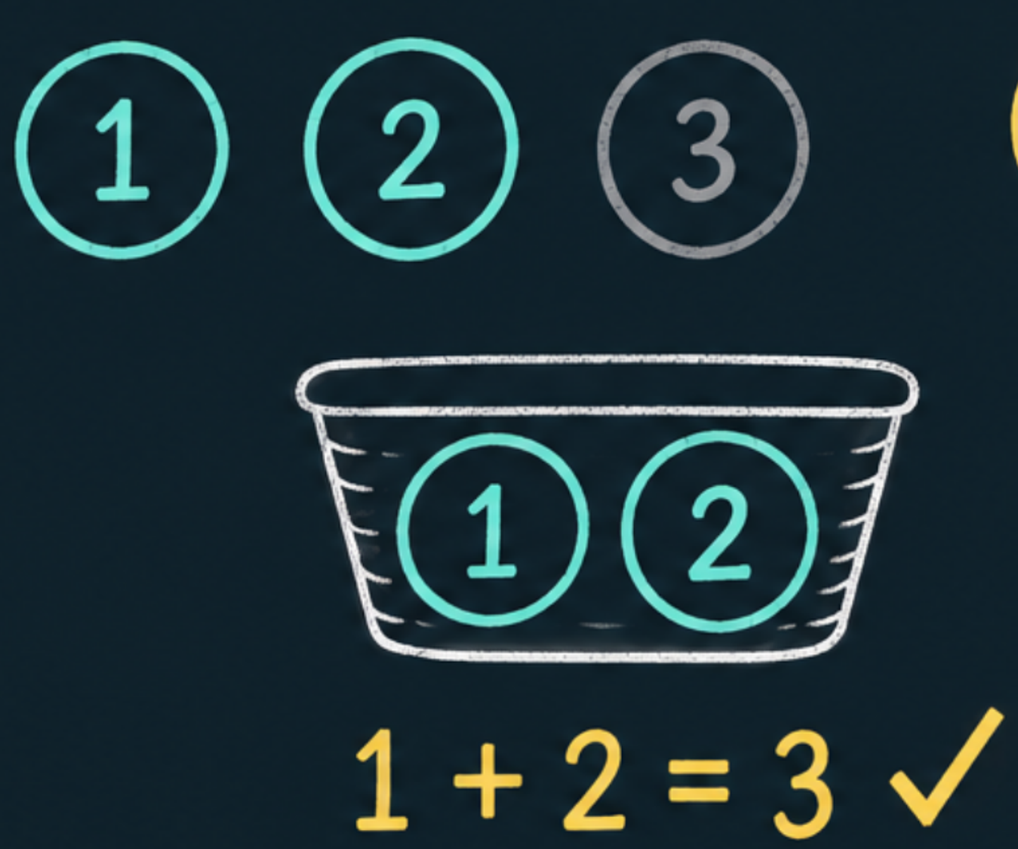
숫자 [1, 2, 3]에서 골라봅니다.

각 위치는 고르거나 건너뜁니다.

고른 순서는 구분하지 않습니다.

조건을 만족하는 선택의 수 반환

1과 2를 고르면 한 가지 성공



선택한 값의 합은 1 + 2 = 3

3은 고르지 않았습니다.

성공한 선택 하나로 셉니다.

현재 성공 수 1

3만 고르는 것도 다른 성공

1과 2는 건너뛰고 3을 고릅니다.

앞의 [1, 2]와 다른 선택입니다.

둘 다 목표 합 3을 만듭니다.

전체 답은 2

한 숫자를 볼 때는 고르기와 건너뛰기

search(i, total): i번부터 결정, 현재 합은 total

건너뛰기: 다음 위치, 같은 합

고르기: 다음 위치, total + a[i]

두 갈래에서 얻은 성공 수를 더합니다.

이미 목표를 넘은 갈래는 멈춥니다

예: [2, 4, 1], 목표 3에서 2와 4를 선택

현재 합 6에 남은 양수를 더해도 3이 될 수 없음

6 > 3이면 이 갈래의 성공 수는 0

이 문제의 모든 원소가 양수라는 조건을 사용합니다.

7-2 · Python

```python
def subset_sum_count(a, target):
    def search(i, total):
        if total > target:
            return 0
        if i == len(a):
            return int(total == target)
        return (search(i+1, total)
                + search(i+1, total+a[i]))
    return search(0, 0)
```

끝까지 결정한 뒤 목표와 같으면 1, 다르면 0을 반환합니다.

7-2 · C

```c
static int subset_search(const int a[], int n, int target,
                         int i, int total) {
    if (total>target) return 0;
    if (i==n) return total==target;
    return subset_search(a,n,target,i+1,total)
         + subset_search(a,n,target,i+1,total+a[i]);
}
int subset_sum_count(const int a[], int n, int target) {
    return subset_search(a,n,target,0,0);
}
```

subset_search는 탐색용 함수, subset_sum_count는 시작 함수입니다.

7-3 · 선으로 연결된 점은 다른 색

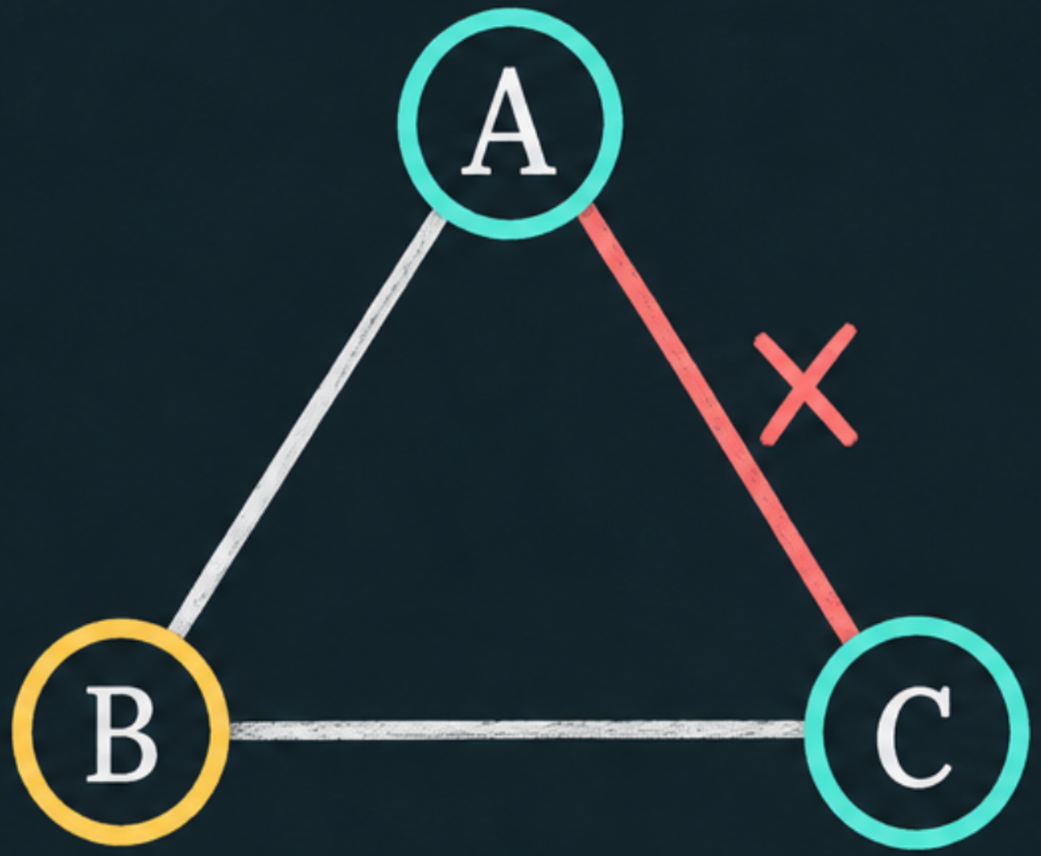
A와 C는 연결되지 않았습니다.

A와 C가 같은 색이어도 됩니다.

A–B, B–C의 양 끝 색만 다르면 됩니다.

가능하면 1, 불가능하면 0

삼각형에는 두 색으로 부족합니다



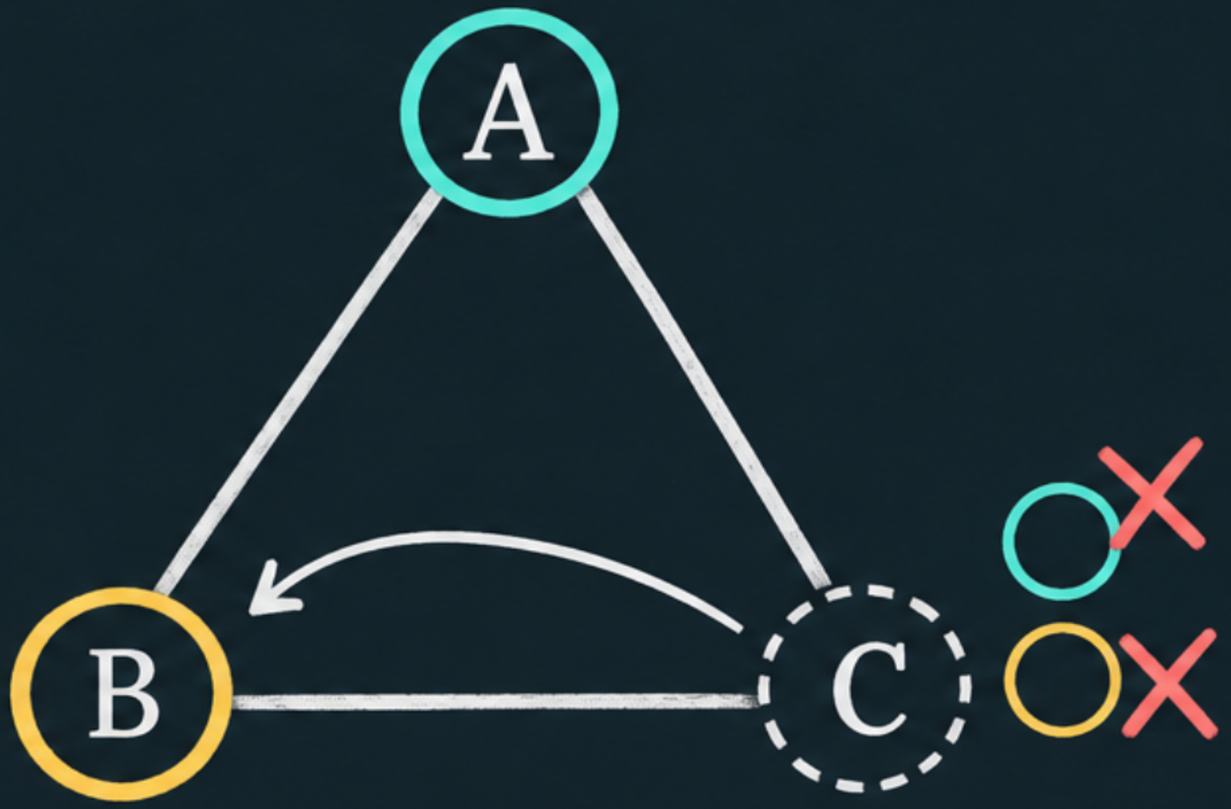
A는 초록, B는 노랑으로 칠했습니다.

C를 초록으로 칠하면 A와 충돌

C를 노랑으로 칠하면 B와 충돌

두 색으로는 완성할 수 없음

다음 점이 막히면 앞의 색을 지웁니다



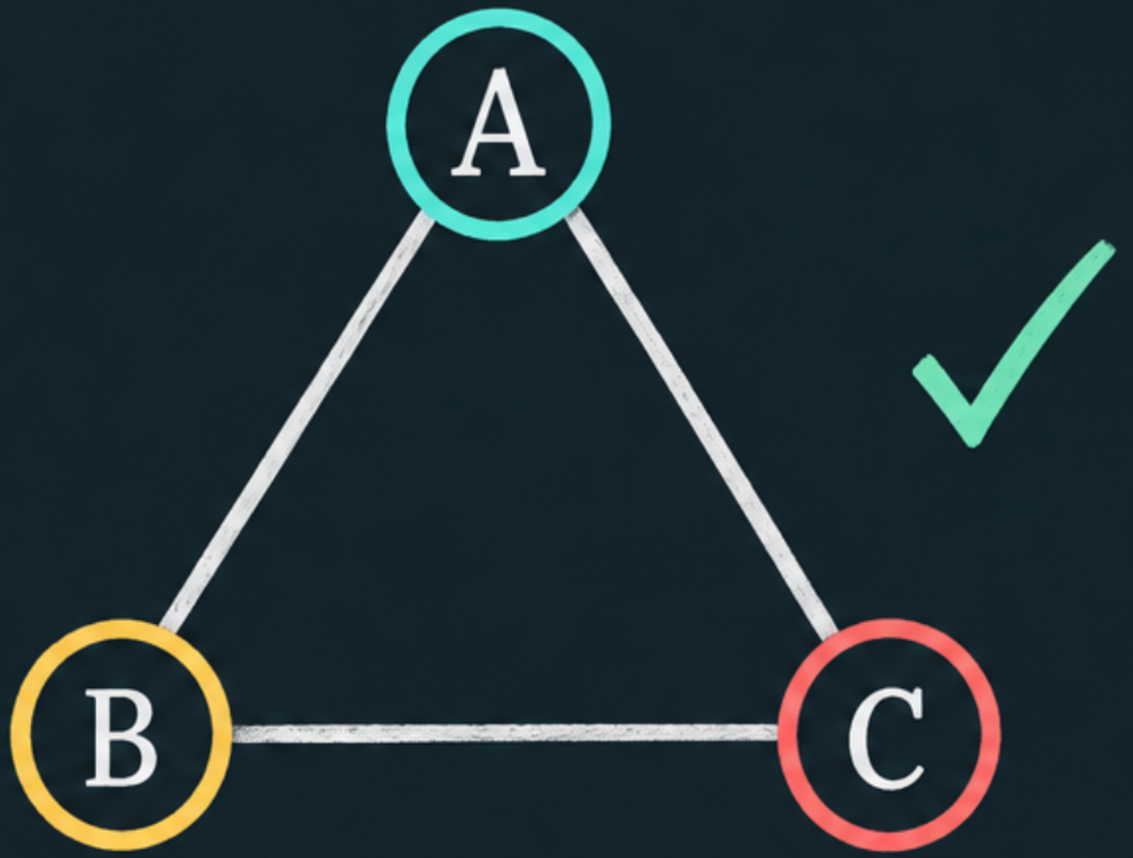
C에서 쓸 수 있는 색이 없습니다.

직전 점 B의 선택으로 돌아갑니다.

B에 다른 색이 가능한지 시험합니다.

모든 색이 실패하면 더 앞으로

세 번째 색이 있으면 완성됩니다



A·B·C가 모두 다른 색입니다.

세 선의 양 끝 색이 모두 다릅니다.

사용 가능한 색을 전부 쓸 의무는 없습니다.

k = 3이면 1 반환

7-3 · Python (1/2)

```python
def can_color(graph, k):
    n = len(graph)
    color = [0]*n
    def paint(v):
        if v == n:
            return 1
        for c in range(1, k+1):
            allowed = True
            for u in range(v):
                if graph[v][u] == 1 and color[u] == c:
                    allowed = False
                    break
```

다음 화면에서 이어집니다.

7-3 · Python (2/2)

```
            if allowed:
                color[v] = c
                if paint(v+1) == 1:
                    return 1
                color[v] = 0
        return 0
    return paint(0)
```

앞 화면과 이어지는 같은 함수입니다.

7-3 · C (1/2)

```
1   static int paint(const int g[][8], int n, int k,
2                    int v, int color[]) {
3       if (v==n) return 1;
4       for (int c=1; c<=k; ++c) {
5           int ok=1;
6           for (int u=0; u<v; ++u)
7               if (g[v][u] && color[u]==c) ok=0;
```

다음 화면에서 이어집니다.

7-3 · C (2/2)

```
8             if (ok) { color[v]=c;
9                 if (paint(g,n,k,v+1,color)) return 1;
10                color[v]=0; }
11        } return 0;
12    }
13    int can_color(const int graph[][8], int n, int k) {
14        int color[8]={0}; return paint(graph,n,k,0,color);
15    }
```

앞 화면과 이어지는 같은 함수입니다.

7-4 · 퀸은 가로·세로·대각선으로 공격

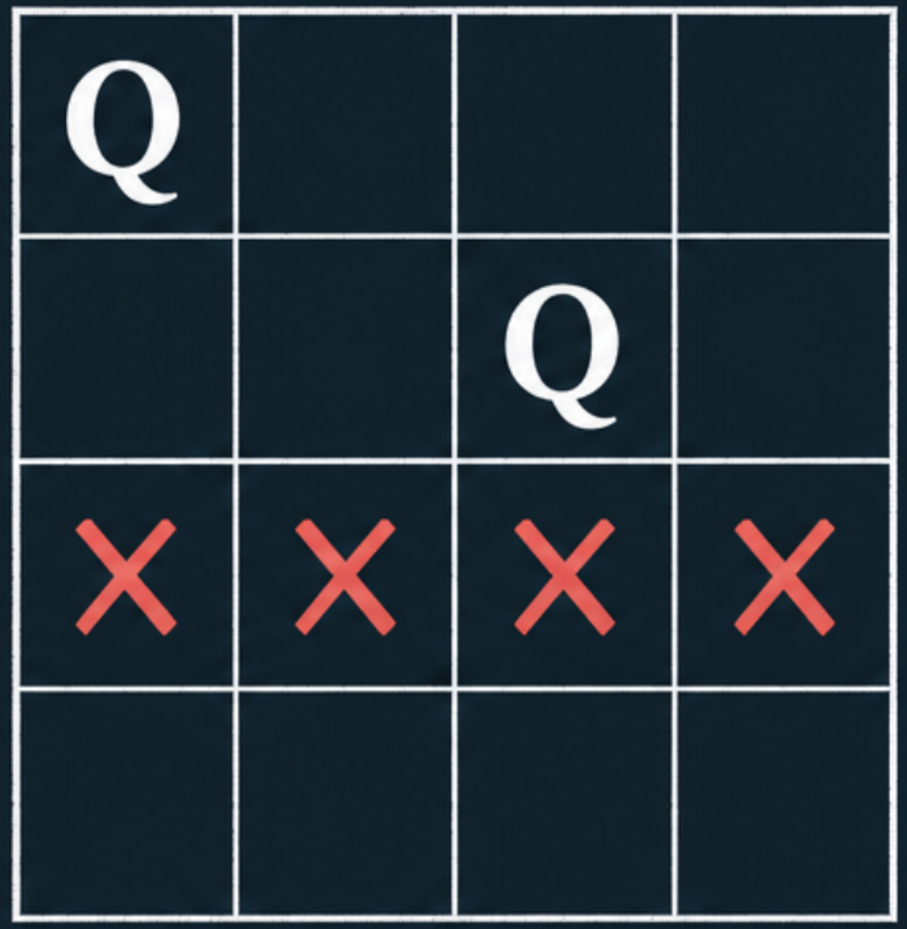
색칠된 칸은 같은 퀸이 공격합니다.

n×n 판에 퀸 n개를 놓습니다.

두 퀸이 서로 공격하면 안 됩니다.

가능한 배치의 개수 반환

첫 선택 때문에 나중 행이 막힐 수 있습니다



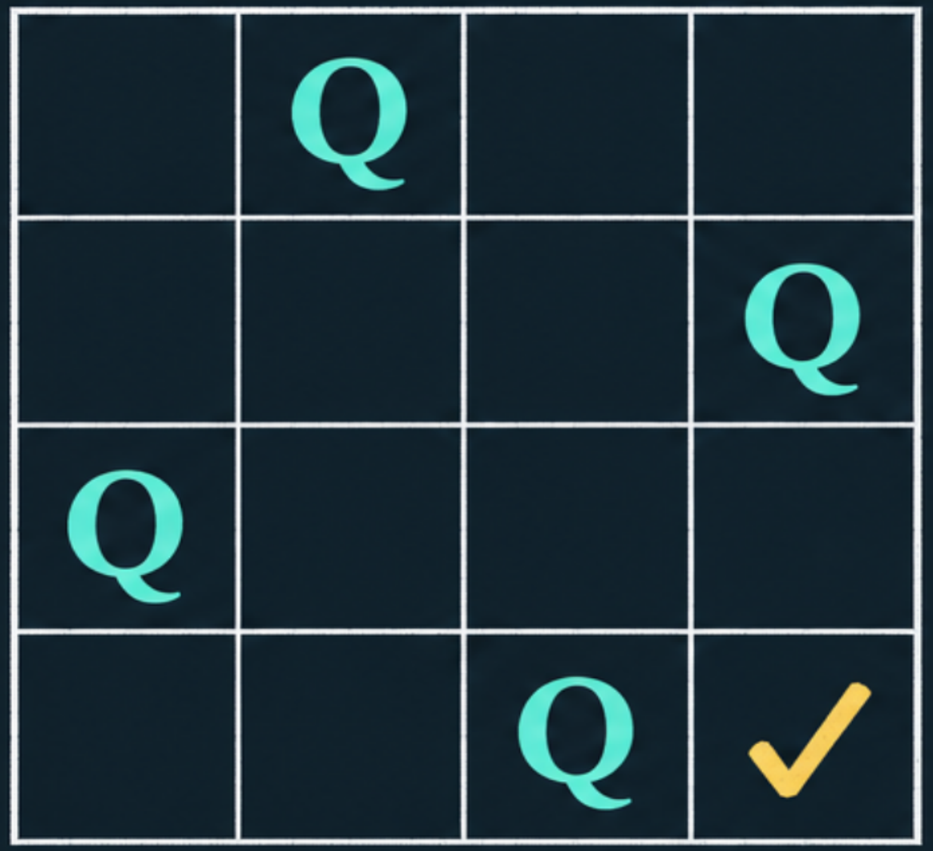
1행 1열, 2행 3열에 놓아봅니다.

3행에는 안전한 칸이 없습니다.

직전 행의 퀸을 빼고 다른 열을 봅니다.

막히면 이전 선택으로 돌아가기

첫 행의 열을 바꾸면 끝까지 놓을 수 있습니다



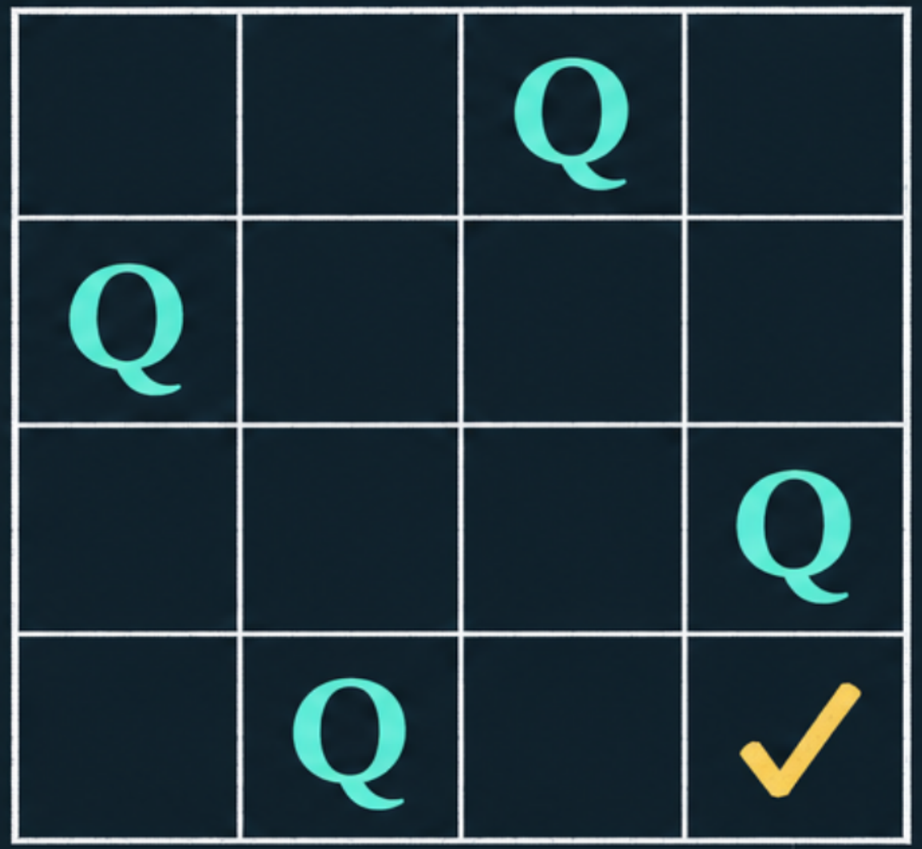
위에서부터 2·4·1·3열

같은 열에 두 퀸이 없습니다.

대각선에도 두 퀸이 겹치지 않습니다.

성공 배치 한 가지

좌우를 뒤집은 배치도 따로 셉니다



위에서부터 3·1·4·2열

앞의 2·4·1·3열과 다른 배치입니다.

4×4 판에서는 두 배치가 전부입니다.

n = 4의 답은 2

대각선이면 행 차이와 열 차이가 같습니다

예: (1행, 2열)과 (2행, 3열)

아래로 1칸, 오른쪽으로 1칸 이동

|열 차이| = 행 차이

같은 열인지 먼저 검사하고, 대각선인지도 검사합니다.

7-4 · Python (1/2)

```python
def nqueen_count(n):
    columns = []
    def place(row):
        if row == n:
            return 1
        answer = 0
        for col in range(n):
            allowed = True
            for r in range(row):
                c = columns[r]
                if col == c or abs(col-c) == row-r:
                    allowed = False
                    break
```

다음 화면에서 이어집니다.

7-4 · Python (2/2)

```
14              if allowed:
15                  columns.append(col)
16                  answer += place(row+1)
17                  columns.pop()
18          return answer
19      return place(0)
```

앞 화면과 이어지는 같은 함수입니다.

7-4 · C

```
1   static int queens(int n, int row, int pos[]) {
2       if (row==n) return 1;
3       int answer=0;
4       for (int col=0; col<n; ++col) {
5           int ok=1;
6           for (int r=0; r<row; ++r) {
7               int d=col-pos[r]; if (d<0) d=-d;
8               if (col==pos[r] || d==row-r) ok=0;
9           }
10          if (ok) {pos[row]=col; answer+=queens(n,row+1,pos);}
11      } return answer;
12  }
13  int nqueen_count(int n) {
14      int pos[8]={0}; return queens(n,0,pos);
15  }
```

pos[row]는 row행에 놓은 퀸의 열입니다.

문제마다 무엇을 반환하는지 먼저 확인합니다

전화 문자: 완성된 문자열 목록

카드·목표 합·퀸: 성공한 경우의 수

색칠: 가능하면 1, 불가능하면 0

성공한 즉시 끝낼지, 모든 성공을 더할지도 달라집니다.

선택과 되돌리기를 직접 따라가며 구현

7-1 카드 배치 수 · 7-2 목표 합 선택 수

7-3 그래프 색칠 가능 여부 · 7-4 퀸 배치 수

지금 선택한 것과 다음 위치를 적기

작은 예를 손으로 풀고, 같은 결과가 나오는지 실행합니다.